

Path Replanning

Path Replanning

1. Course Content
2. Edit Road Network
3. Run Path Replanning Case
 - 3.1 Start Agent
 - 3.2 Start Sensors and Odometry
 - 3.3 Start Road Network Planning and Navigation Node
4. Query Obstacle Cost Threshold
 - 4.1 View Obstacle Cost Values
 - 4.2 Modify Threshold for Triggering Replanning
5. Source Code Analysis
 - 5.1 Obstacle Detection and Replanning Strategy
 - 5.2 Query Map Cost Value Implementation Method

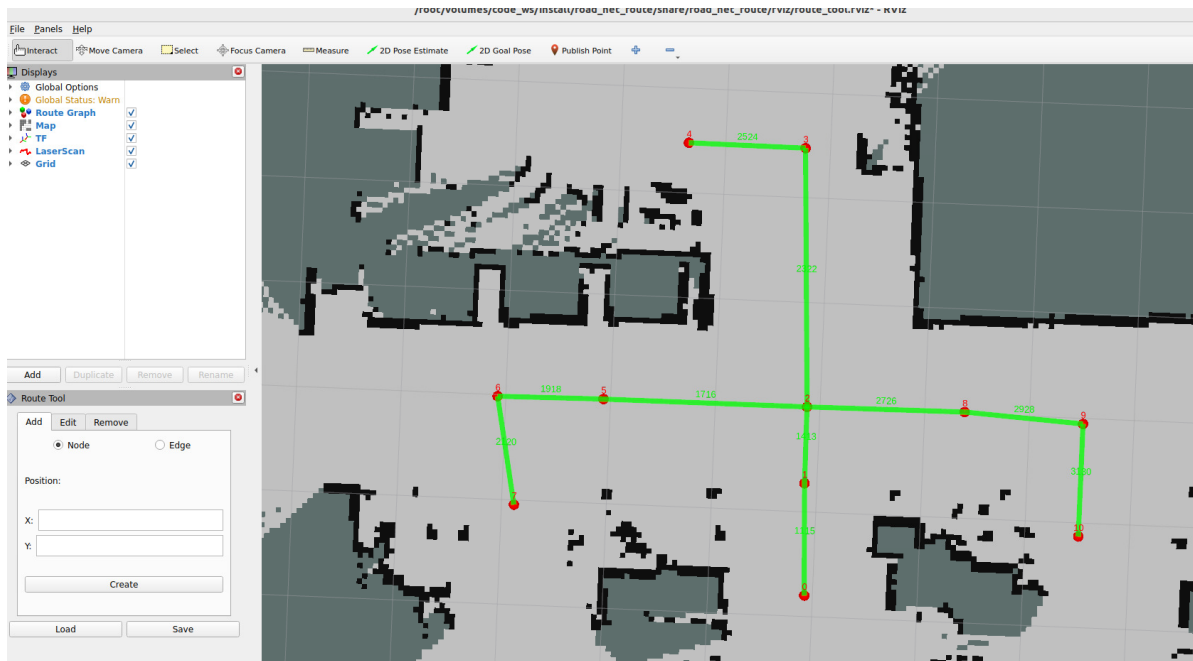
1. Course Content

- Basic

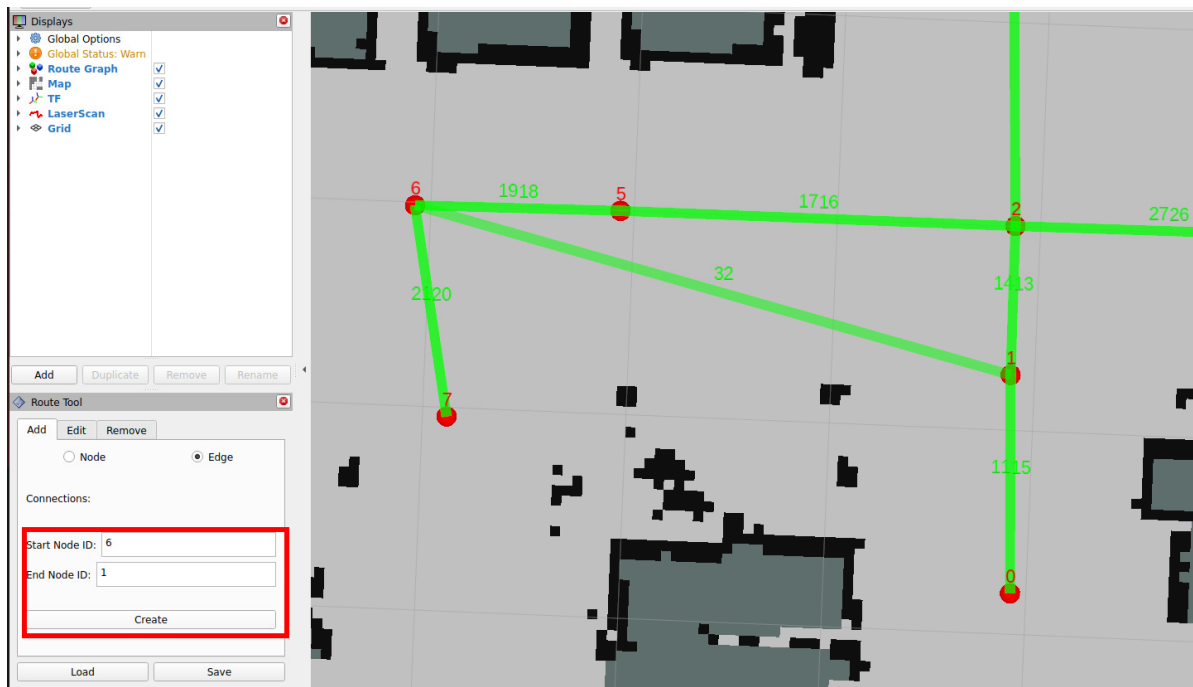
Run the example program. On the basis of road network planning and navigation, if it is detected that the driving path is blocked during driving, it will enter replanning to find a new path in the road network
- Advanced
 1. Understand the implementation method of replanning and how to change the replanning strategy (strategies are not unique, you can implement them according to your needs)
 2. Debug and change obstacle detection thresholds according to queried cost values

2. Edit Road Network

- To demonstrate the replanning function, we add some edges to the road network in the previous course example. For opening and editing road network operations, refer to: Section 4 of **03-Road Network Annotation**: Loading and Modifying Road Network Description



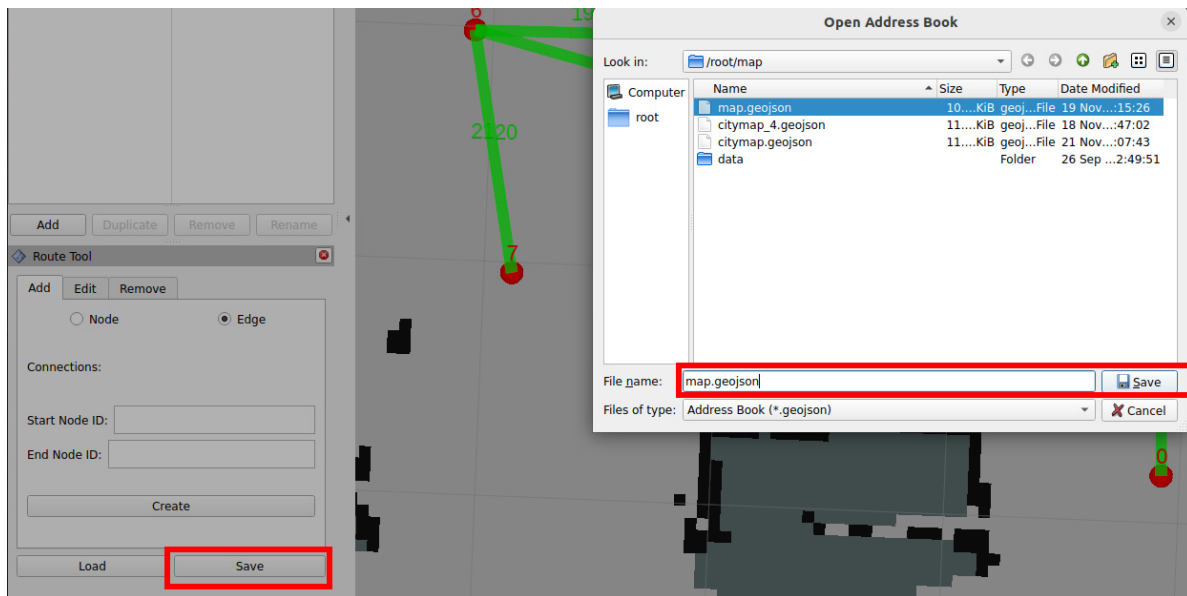
- For example, add bidirectional edges between waypoints 1 and 6



- All operations will have corresponding records in the terminal

```
[rviz2-1] [INFO] [1764245498.392310926] [route_tool_node]: Adding edge from 1 to 6
[rviz2-1] [INFO] [1764245596.474192853] [route_tool_node]: Adding edge from 6 to 1
```

- Then save the file



3. Run Path Replanning Case

3.1 Start Agent

- Use the following command to start the microROS agent and connect to the chassis

[!WARNING]

The commands for starting the agent, sensors and odometry, and road network planning navigation nodes here are the same as the previous lesson. If already started, **no need to start again**

```
start_kilted_agent
```

```
jetson@yahboom: ~
jetson@yahboom: ~ 80x24
subscriber created | client_key: 0x14F22590, subscriber_id: 0x000(4), participant_id: 0x000(1)
[1763535321.445792] info | ProxyClient.cpp | create_datareader | d
datareader created | client_key: 0x14F22590, datareader_id: 0x000(6), subscriber_id: 0x000(4)
[1763535321.449556] info | ProxyClient.cpp | create_topic | t
topic created | client_key: 0x14F22590, topic_id: 0x006(2), participant_id: 0x000(1)
[1763535321.452876] info | ProxyClient.cpp | create_subscriber | s
subscriber created | client_key: 0x14F22590, subscriber_id: 0x001(4), participant_id: 0x000(1)
[1763535321.457712] info | ProxyClient.cpp | create_datareader | d
datareader created | client_key: 0x14F22590, datareader_id: 0x001(6), subscriber_id: 0x001(4)
[1763535321.461959] info | ProxyClient.cpp | create_topic | t
topic created | client_key: 0x14F22590, topic_id: 0x007(2), participant_id: 0x000(1)
[1763535321.466141] info | ProxyClient.cpp | create_subscriber | s
subscriber created | client_key: 0x14F22590, subscriber_id: 0x002(4), participant_id: 0x000(1)
[1763535321.473112] info | ProxyClient.cpp | create_datareader | d
datareader created | client_key: 0x14F22590, datareader_id: 0x002(6), subscriber_id: 0x002(4)
```

- Modify ROS communication middleware DDS

[!WARNING]

For Raspberry Pi 5 users, if the ROS container is not started, you need to start the container first,

```
bringup_m3
```

Open a terminal inside the container,

```
exec_m3
```

Open ~/.bashrc, uncomment this line, and change it to the DDS communication middleware as shown in the figure (only the road network planning and navigation chapter needs to use cyclondds, **please restore the environment variables after completing the road network chapter!!**)

```
vim ~/.bashrc
```

- RDK & PI5 mainboards

```
131 #ROSMaster-M3 Code space environment variables
132 export PYTHONPATH="/home/sunrise/yahboomM3_ws/.venv/lib/python3.10/site-packages:$PYTHONPATH"
133 export ROS_DOMAIN_ID=16
134 export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
135 #unset RMW_IMPLEMENTATION
136 export ROBOT_TYPE="ROSMaster-M3"
137 export MULTI_ROBOT="" #robot1 or robot2
138 #=====雷达和相机型号=====
139 #=====Radar and camera models=====
140 #=====
141 #单雷达: single          双雷达: dual
142 #Single radar: single    Dual radar: dual
143 export LASER_TYPE=single
144 #export LASER_TYPE=dual
145
```

- ORIN mainboard

```
133 export ROS_DOMAIN_ID=30
134 export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
135 export ROBOT_TYPE="ROSMaster-M3"
136 export MULTI_ROBOT=""
137
```

```
:wq #Save and exit
source ~/.bashrc #Update environment
```

3.2 Start Sensors and Odometry

- Open a terminal on the host to start the vehicle's radar fusion node and odometry fusion
- (For PI5 users, start in the `m3` container)

```
ros2 launch m3_bringup car_base.launch.py
```

```

jetson@yahboom:~$ ros2 launch m3_bringup car_base.launch.py
[INFO] [launch]: All log files can be found below /home/jetson/.ros/log/2025-11-19-18-43-09-279818-yahboom-1129502
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [launch.user]: =====Robot model ROSMASTER-M3 start=====
                        =====LASER_TYPE:single=====

[INFO] [launch.user]: LASER start sucefully!,Start by independent node
[INFO] [launch.user]: =====Robot Base Start=====
[INFO] [joint_state_publisher-1]: process started with pid [1129557]
[INFO] [robot_state_publisher-2]: process started with pid [1129559]
[INFO] [lasers_merger-3]: process started with pid [1129561]
[INFO] [scan_to_scan_filter_chain-4]: process started with pid [1129564]
[lasers_merger-3] [INFO] [1763548989.905313662] [laserscan_multi_merger]: Waiting for topics ...
[robot_state_publisher-2] [INFO] [1763548989.924917431] [robot_state_publisher]: got segment ascamera_hp60c_camera
link_0
[robot_state_publisher-2] [INFO] [1763548989.925142333] [robot_state_publisher]: got segment base_footprint
[robot_state_publisher-2] [INFO] [1763548989.925167773] [robot_state_publisher]: got segment base_link
[robot_state_publisher-2] [INFO] [1763548989.925181310] [robot_state_publisher]: got segment camera_link
[robot_state_publisher-2] [INFO] [1763548989.925191422] [robot_state_publisher]: got segment imu_frame
[robot_state_publisher-2] [INFO] [1763548989.925200830] [robot_state_publisher]: got segment laser23_frame
[robot_state_publisher-2] [INFO] [1763548989.925209982] [robot_state_publisher]: got segment lwheel1_Link
[robot_state_publisher-2] [INFO] [1763548989.925218911] [robot_state_publisher]: got segment lwheel2_Link
[robot_state_publisher-2] [INFO] [1763548989.925227999] [robot_state_publisher]: got segment rwheel1_Link
[robot_state_publisher-2] [INFO] [1763548989.925236735] [robot_state_publisher]: got segment rwheel2_Link
[scan_to_scan_filter_chain-4] [WARN] [1763548989.935781454] [rcl.logging_rosout]: Publisher already registered for
provided node name. If this is due to multiple nodes with the same name then all logs for that logger name will g
o out over the existing publisher. As soon as any node with that name is destructed it will unregister the publish
er, preventing any further logs for that name from being published on the rosout topic.
[joint_state_publisher-1] [INFO] [1763548990.531349805] [joint_state_publisher]: Waiting for robot_description to
be published on the robot_description topic...
[lasers_merger-3] [INFO] [1763548990.907903719] [laserscan_multi_merger]: Subscribing to topics 2
[scan_to_scan_filter_chain-4] [WARN] [1763548990.924837083] [laser_filter]: diagnostic_updater: No HW_ID was set.
This is probably a bug. Please report it. For devices that do not have a HW_ID, set this value to 'none'. This war
ning only occurs once all diagnostics are OK. It is okay to wait until the device is open before calling setHardwa
reID.
[INFO] [launch.user]: =====Merge Odom Localization Start=====
[INFO] [ekf_node-5]: process started with pid [1129717]

```

3.3 Start Road Network Planning and Navigation Node

- If the ros2_kilted docker container is not started, you need to start the container first

```
bringup_ros2kilted
```

- Open a terminal inside the container

```
exec_ros2kilted
```

- (PI5 & RDK mainboards) Visualization is started on the accompanying virtual machine
Rosmaster-RoadNet-Ubuntu24.04 ,

```
rviz2 -d /home/yahboom/code_ws/src/road_net_route/rviz/nav2_view.rviz
```

- Start road network planning and navigation node

```
ros2 launch road_net_route roadnet_nav2.launch.py
```

[!TIP]

Optional startup parameter description

map The grid map file name to be loaded, default value is `map.yaml`

pose_map The pose map file name to be loaded. When the navigation global positioning method uses slam_toolbox pure localization, the pose map must be passed, default value is

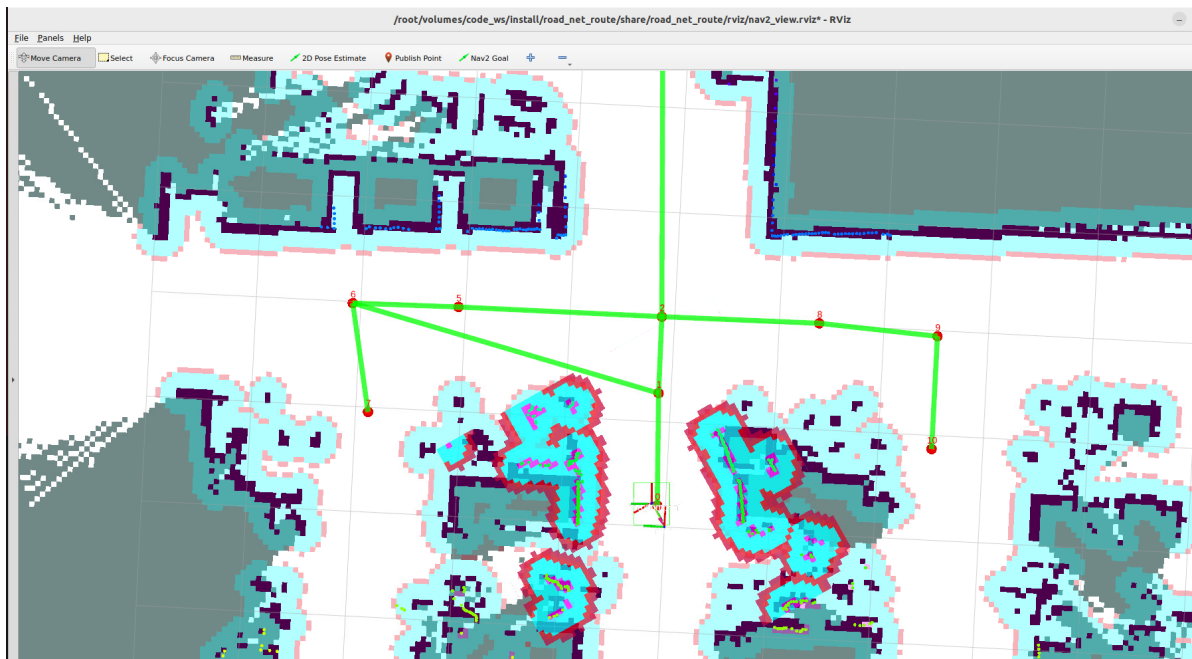
`map`

params_file Navigation parameter file, default value is

```
os.path.join(get_package_share_directory('road_net_route'), 'config',  
'nav2_kilted.yaml')
```

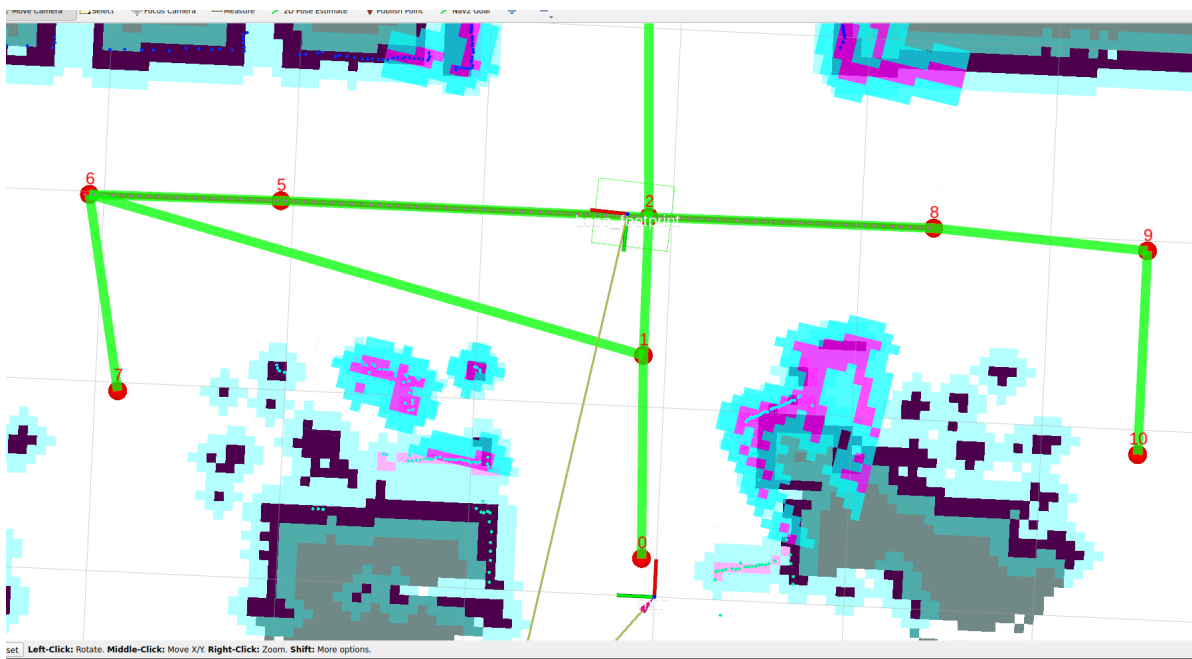
roadnet_file Road network description file, default value is `map.geojson`

localization_mode Global positioning method, default is `slam_toolbox`



- The test scenario here is: travel from waypoint 8 along the road network to waypoint 6, then place obstacles on the path planned by the road network to trigger the replanning strategy

```
ros2 topic pub -1 /road_net_nav_id std_msgs/msg/Int16MultiArray "{data: [8, 6]}"
```



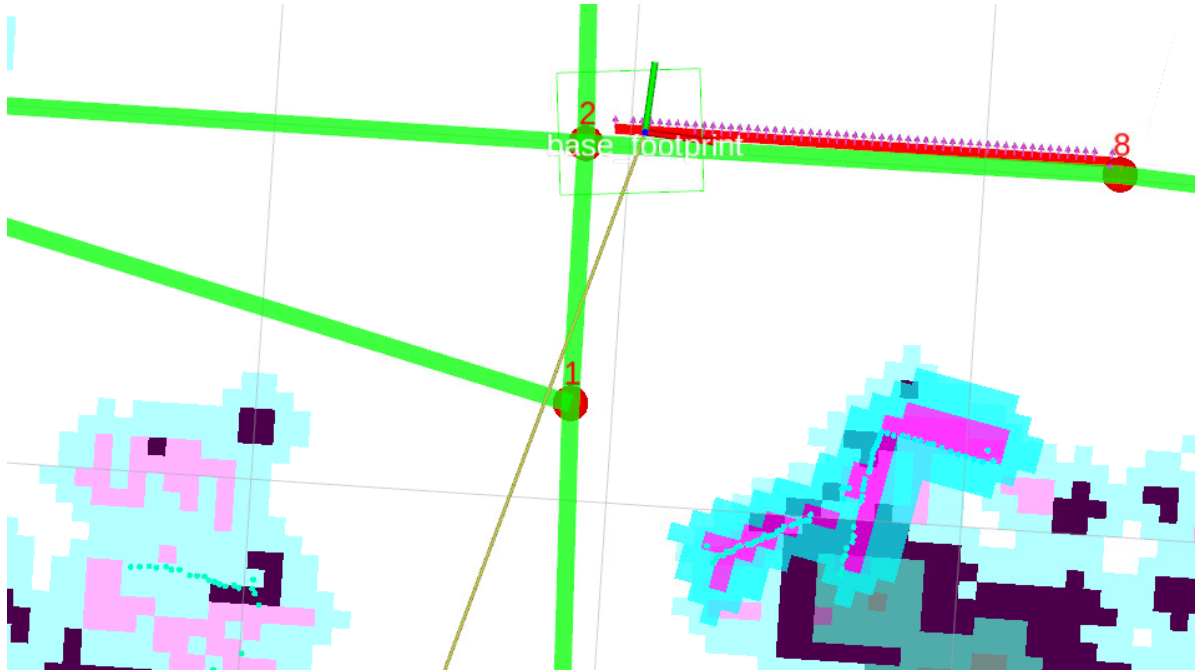
- Here you can see the terminal will prompt that there are obstacles on the path, then trigger replanning


```

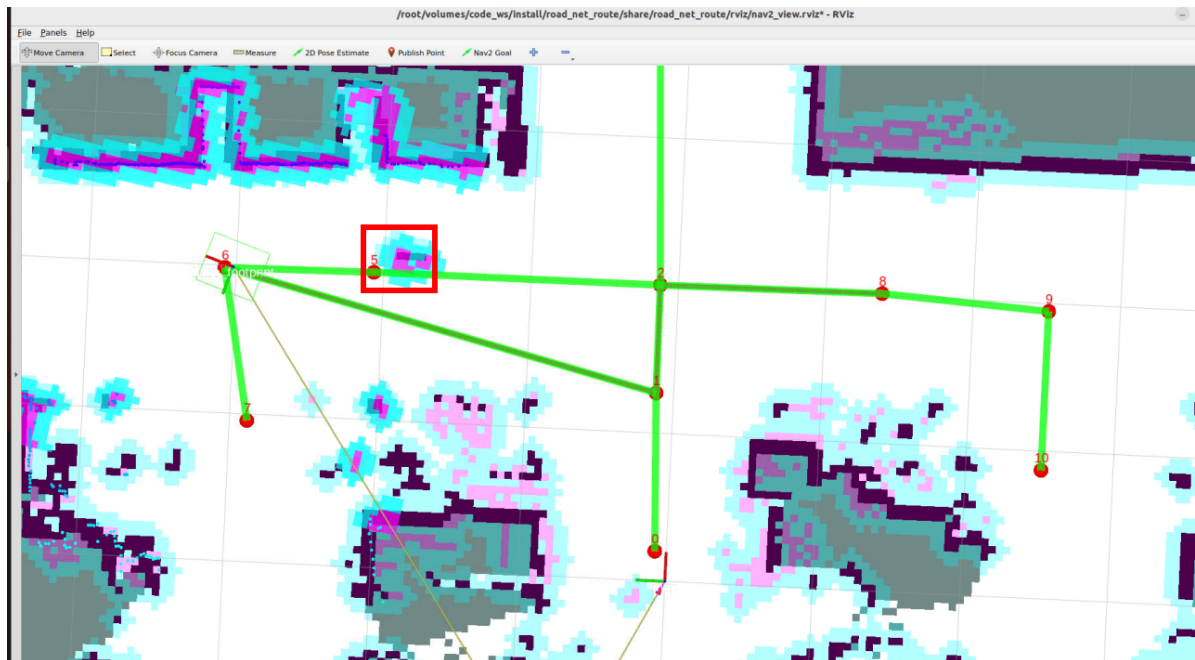
loop rate 0.1 20.0000 Hz. current loop rate is 15.7802 Hz.
[route_bridge-4] [INFO] [1764248055.001985898] [route_bridge]: id:2 -----> id:5 edge:16
[component_container_isolated-1] [INFO] [1764248055.687587193] [global_costmap.global_costmap]: StaticLayer: Resizing costmap to 225 X 599 at 0.050000 m/pix
[route_bridge-4] [INFO] [1764248057.758462756] [route_bridge]: Path contains 7 potential obstacles.
[route_bridge-4] [INFO] [1764248058.249064867] [route_bridge]: Navigating to goal: 2.0027759075164795 0.1238360032 4392319...

```

- Here it is detected that there are obstacles on the path, after the path is impassable, retreat to the last passed waypoint 8



- Then retry to plan a drivable path from waypoint 8, then reach the target waypoint 6



4. Query Obstacle Cost Threshold

- Here the detection of obstacles on the path is determined by detecting the cost value on the global path and the number of points with high cost values in the global path. The threshold for triggering replanning can be set by yourself

[!NOTE]

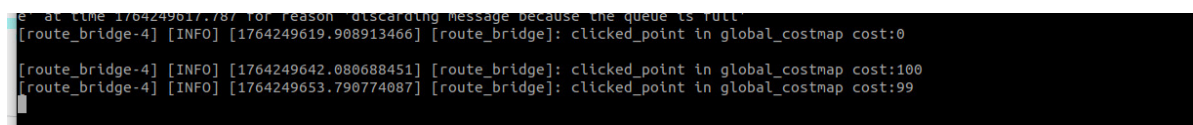
- The obstacle detection threshold generally uses the factory default!!! If there are special scenario requirements, you can refer to this part of the tutorial to adjust the threshold for triggering replanning
- Cost value is the value used in the cost map to describe the occupied range of obstacles. The closer to the obstacle, the higher the cost value.

4.1 View Obstacle Cost Values

- Click the Publish Point tool, then use the left mouse button to click on the map where you want to view the map cost value, you can view the cost value of the specified position



- You can see that the cost value is 0 where there are no obstacles, and the cost value at the obstacle center is 99-100



4.2 Modify Threshold for Triggering Replanning

- Configuration file path

```
/home/jetson/yahboomM3_ws/ros2_kilted/volumes/code_ws/src/road_net_route/config/
nav2_kilted.yaml
```

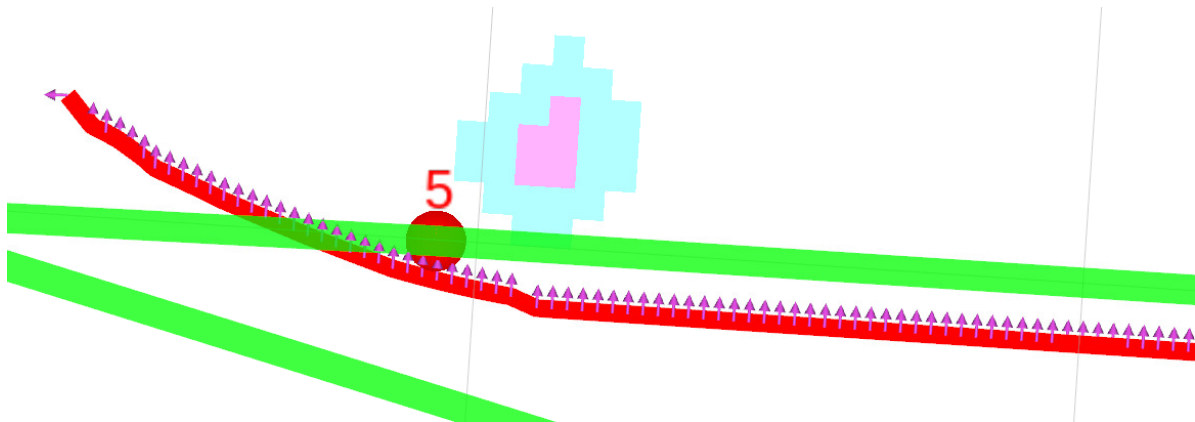
- Find the route_bridge node configuration parameters in the configuration file, where `COST_THRESHOLD` and `PATH_OBSTACLES_THRESHOLD` jointly determine the trigger threshold


```
route_bridge:
  ros__parameters:
    roadnet_file: '/root/map/map.geojson'
    COST_THRESHOLD: 60
    replace2id: True
    PATH_OBSTACLES_THRESHOLD: 5
```

- Specific obstacle detection logic is as follows
- The global path planned by the road network function is the `nav_msgs/msg/Path` message type in ROS, which is a path composed of multiple pose points. During road network navigation, it will detect whether all pose points on the global path have points with high cost values in the global cost map.
 - If the cost value of a certain pose point on the path exceeds the **COST_THRESHOLD** threshold, mark that point on the path as a high cost value pose point
 - Finally calculate how many pose points on the global are cost value pose points
 - If the total number of high cost value pose points is greater than **PATH_OBSTACLES_THRESHOLD**, then this path is considered impassable and needs replanning

[!TIP]

If obstacle detection is not sensitive enough, you can reduce **COST_THRESHOLD** and **PATH_OBSTACLES_THRESHOLD**. If it is too sensitive, you can reduce these two values or adjust the obstacle inflation radius (for modifying obstacle inflation radius, refer to 4.1 Navigation Controller Debugging in 04-Inter-Road Network Planning and Navigation)



5. Source Code Analysis

- Implementation program source code path:

```
/home/jetson/yahboomM3_ws/ros2_kilted/volumes/code_ws/src/road_net_route/road_net_route/route_bridge.py
```

5.1 Obstacle Detection and Replanning Strategy

- Traverse each pose point of the global path, check if there are obstacles exceeding the threshold on the path through the global cost map, return a boolean value indicating whether the path is blocked by obstacles.
- **World coordinates to map coordinates:** Call the cost map's `worldToMapValidated` method to convert world coordinate system `(wx, wy)` to cost map's **grid coordinates** `(mx, my)` (cost map is a 2D grid map, each grid corresponds to a cost value).
- **Coordinate validity check:** If the converted grid coordinates are `None`, it means this path point is outside the global cost map range, print warning log and skip this point.
- **Parameters:** `path:Path` → ROS2's `nav_msgs/msg/Path` message type, stores the robot's global planned path, contains a series of poses (`poses`).
- **Return value:** `bool` → `True` means the number of obstacles on the path exceeds the threshold, `False` means the path is feasible.

```
def __check_path_obstacles(self, path:Path)->bool:
    """Check for obstacles on the global path."""
    pose_array=[]
    for pose in path.poses:
        wx=pose.pose.position.x
        wy=pose.pose.position.y

        mx, my = self.global_costmap.worldToMapValidated(wx,wy )
        if mx is None or my is None:
            self.get_logger().warn(f"Path point ({wx}, {wy}) is outside global
costmap")
            continue

        cost = self.global_costmap.getCostXY(int(mx),int(my))

        if cost > self.COST_THRESHOLD:
            pose_array.append(pose)
    if len(pose_array) >self.PATH_OBSTACLES_THRESHOLD:
        self.get_logger().info(f"Path contains {len(pose_array)} potential
obstacles.")
        return True
    return False
```

- When obstacles are detected on the global path, let the robot **go back to the previous waypoint** and replan the path.

```
def __reroute_strategy_1(self,last_id):
    '''Replanning Strategy 1: Go back to the previous waypoint and replan the
drivable path.'''
    navigator= BasicNavigator()
    goal_x,goal_y=self.__get_node_coordinates(last_id)
    if goal_x is not None and goal_y is not None:
        goal_pose = PoseStamped()
        goal_pose.header.frame_id = "map"
        goal_pose.pose.position.x = goal_x
        goal_pose.pose.position.y = goal_y
        back_lastid = navigator.goToPose(goal_pose)
        while not navigator.isTaskComplete(task=back_lastid):
```

```

        time.sleep(0.1)
    if navigator.getResult() == TaskResult.SUCCEEDED:
        self.get_logger().info("back to last id success.")
    else:
        self.get_logger().info("back to last id failed.")

```

- `handle_id_navigation` and `handle_pose_navigation` are the core code responsible for road network planning and navigation. The following parts in these two functions are the logic for detecting paths and triggering replanning

When the path is detected as infeasible, call **Replanning Strategy 1**: Let the robot **go back to the previous waypoint** (coordinates corresponding to `last_node_id`). The core logic of this method has been explained before: initialize navigator → get coordinates of `last_node_id` → send navigation task "go back to this waypoint" → wait for task completion and print results. Move the robot from the current obstacle area back to **safe predefined waypoint**, providing a stable starting point for subsequent path replanning.

The complete process is as follows:

1. Robot normally executes path tracking, gets navigation feedback `feedback` in real time.
2. Detect whether there are obstacles on the path in `feedback`:
 - If no obstacles: continue executing the original path, no processing needed.
 - If there are obstacles: trigger replanning logic.
3. Robot goes back to the previous safe waypoint (`last_node_id`).
4. Use this waypoint as the starting point to replan the path to the target waypoint (`goal_id`).
5. Get feedback of the new path, let the robot track the new path and continue moving.

```

if feedback is not None and self.__check_path_obstacles(feedback.path):
    self.__reroute_strategy_1(last_node_id)
    route_task = self.navigator.getAndTrackRoute(last_node_id, goal_id)
    feedback = self.navigator.getFeedback(task=route_task)
    follow_path_task = self.navigator.followPath(feedback.path)

```

```

def handle_id_navigation(self, start_id:int, goal_id:int):
    '''Road network ID navigation'''

    route_task = self.navigator.getAndTrackRoute(start_id, goal_id)
    feedback=None
    follow_path_task=RunningTask.NONE
    last_feedback = None
    last_node_id =999
    while not self.navigator.isTaskComplete(task=route_task):
        feedback = self.navigator.getFeedback(task=route_task)
        if feedback is not None and
self.__check_path_obstacles(feedback.path):
            self.__reroute_strategy_1(last_node_id)
            route_task = self.navigator.getAndTrackRoute(last_node_id,
goal_id)

```

```

        feedback = self.navigators.getFeedback(task=route_task)
        follow_path_task = self.navigators.followPath(feedback.path)
        while feedback is not None:
            if last_feedback is not None and (feedback.last_node_id !=
last_feedback.last_node_id or feedback.next_node_id !=
last_feedback.next_node_id):
                self.get_logger().info(f'id:{feedback.last_node_id} ----->
id:{feedback.next_node_id} edge:{feedback.current_edge_id}')
                last_feedback = feedback
                last_node_id=feedback.last_node_id
                if feedback.rerouted :
                    self.get_logger().info('reroute a new path to follow!')
                    follow_path_task = self.navigators.followPath(feedback.path)
                    feedback = self.navigators.getFeedback(task=route_task)

            if self.navigators.isTaskComplete(task=follow_path_task):
                print(follow_path_task)
                self.get_logger().info('Controller completed its task!')
                if follow_path_task != RunningTask.NONE :
                    self.navigators.cancelTask()

        result = self.navigators.getResult()
        if result == TaskResult.SUCCEEDED:
            self.get_logger().info(Fore.GREEN+"navigation task
completed."+Fore.RESET)

        self.action_feedback_pub.publish(String(data="road_net_nav_succeeded"))
        elif result == TaskResult.CANCELED:
            self.get_logger().warn("Navigation task canceled.")
        elif result == TaskResult.FAILED:
            self.get_logger().error("Navigation task failed.")
            self.action_feedback_pub.publish(String(data="road_net_nav_failed"))

    def handle_pose_navigation(self, goal_pose:PoseStamped):
        '''Handle arbitrary pose navigation'''
        transform = self.tf_buffer.lookup_transform("map", "base_footprint",
rclpy.time.Time(),rclpy.duration.Duration(seconds=5.0))
        if transform is None:
            self.get_logger().info(Fore.RED+"Failed to get TF transform
"+Fore.RESET)
            return

        else:
            start_pose = PoseStamped()
            start_pose.header = transform.header
            start_pose.pose.position.x = transform.transform.translation.x
            start_pose.pose.position.y = transform.transform.translation.y
            start_pose.pose.orientation = transform.transform.rotation
            self.get_logger().info(Fore.CYAN + f"current:
{transform.transform.translation}"+Fore.RESET)
            nearest_start_node_id, nearest_start_distance =
self.__find_nearest_node(start_pose.pose.position.x, start_pose.pose.position.y)
            nearest_goal_node_id, nearest_goal_distance =
self.__find_nearest_node(goal_pose.pose.position.x, goal_pose.pose.position.y)
            if self.replace2id:
                self.get_logger().info(Fore.CYAN + f"nearest_start_node_id:
{nearest_start_node_id}, distance:{nearest_start_distance}"

```

```

                                f"\nnearest_goal_node_id:
{nearest_goal_node_id}, distance:{nearest_goal_distance}"+Fore.RESET)
        route_task =
self.navigators.getAndTrackRoute(start=nearest_start_node_id,goal=nearest_goal_no
de_id)

        else:
            route_task =
self.navigators.getAndTrackRoute(start=start_pose,goal=goal_pose,use_start=True)

            feedback=None
            follow_path_task=RunningTask.NONE
            last_feedback = None
            last_node_id =999
            while not self.navigators.isTaskComplete(task=route_task):
                feedback = self.navigators.getFeedback(task=route_task)
                if feedback is not None and
self.__check_path_obstacles(feedback.path):
                    self.__reroute_strategy_1(last_node_id)
                    route_task = self.navigators.getAndTrackRoute(last_node_id,
nearest_goal_node_id)
                    feedback = self.navigators.getFeedback(task=route_task)
                    follow_path_task = self.navigators.followPath(feedback.path)
                    while feedback is not None:
                        if last_feedback is not None and (feedback.last_node_id !=
last_feedback.last_node_id or feedback.next_node_id !=
last_feedback.next_node_id):
                            self.get_logger().info(f'id:{feedback.last_node_id} ----
--> id:{feedback.next_node_id} edge:{feedback.current_edge_id}')
                            last_feedback = feedback
                            last_node_id=feedback.last_node_id
                            if feedback.rerouted :
                                self.get_logger().info('reroute a new path to follow!')
                                follow_path_task =
self.navigators.followPath(feedback.path)
                                feedback = self.navigators.getFeedback(task=route_task)

                                if self.navigators.isTaskComplete(task=follow_path_task):
                                    self.get_logger().info('Controller completed its task!')
                                    if follow_path_task != RunningTask.NONE :
                                        self.navigators.cancelTask()

                                result = self.navigators.getResult()
                                if result == TaskResult.SUCCEEDED:
                                    if self.__adjust_pose(goal_pose):
                                        self.get_logger().info(Fore.GREEN+"navigation task
completed."+Fore.RESET)

                                    self.action_feedback_pub.publish(String(data="road_net_nav_succeeded"))

                                elif result == TaskResult.CANCELED:
                                    self.get_logger().info(Fore.YELLOW+"navigation task
canceled."+Fore.RESET)

                                elif result == TaskResult.FAILED:
                                    self.get_logger().info(Fore.RED+"navigation task
failed."+Fore.RESET)

                                    self.action_feedback_pub.publish(String(data="road_net_nav_failed"))

```

5.2 Query Map Cost Value Implementation Method

- Call the global cost map's `worldToMapValidated` method to convert **world coordinates** (`x`, `y`) to **cost map's grid coordinates** (`mx`, `my`).
- Cost map is a **2D grid map** (each grid corresponds to a small area in the real world, 0.05m×0.05m), `worldToMapValidated` is a safe conversion method: if coordinates exceed the cost map range, return `None`, avoiding out-of-bounds errors caused by direct conversion.
- **Get cost value:** Call the cost map's `getCostXY` method, pass in the integer form of grid coordinates (grid coordinates are discrete integers, avoid errors caused by passing in floating-point type), get the **cost value** `cost` of this grid.

```
def get_costmap_value(self, msg: PointStamped):
    '''In RViz, you can click to view the cost of a point on the global cost
    map.'''
    x=msg.point.x
    y=msg.point.y
    mx, my = self.global_costmap.worldToMapValidated(x, y)
    if mx is None or my is None:
        self.get_logger().warn(f"Path point ({x}, {y}) is outside global
    costmap")
    return
    cost = self.global_costmap.getCostXY(int(mx),int(my))
    self.get_logger().info(f"clicked_point in global_costmap cost:{cost}")
```